# DOCUMENTATION TECHNIQUE

## Application MAINTANO

**Projet : Suivi d'entretien du parc d'engins de chantier MAINTANO**

---

### Introduction

Ce document présente la documentation technique détaillée de l'application de gestion de parc d'engins de chantier développée pour l'entreprise **MAINTANO**. L'application, développée en **Lazarus (Pascal)** avec une base de données **SQLite** locale, permet le suivi des véhicules, des points de contrôle et des tâches d'entretien.

Conformément au cahier des charges, cette documentation détaille pour chaque fenêtre de l'application la liste précise des fonctionnalités offertes ainsi que la liste des composants utilisés et la motivation de leur choix.

---

### 0. Fenêtre Principale de Navigation (Unit1)

#### Fonctionnalités

Cette fenêtre constitue la structure globale et le point d'entrée de l'application. Elle gère la navigation générale et les connexions globales de l'application :

- **Navigation centralisée** : Permet de basculer de manière fluide entre les différents modules de l'application (Accueil, Véhicules, Points de contrôle, Statistiques).
- **Affichage dynamique** : Charge et affiche dynamiquement l'interface de chaque module sélectionné au sein d'un conteneur central unique.
- **Gestion globale de la persistance** : Centralise la connexion physique et la gestion transactionnelle de la base de données SQLite pour l'ensemble du projet.

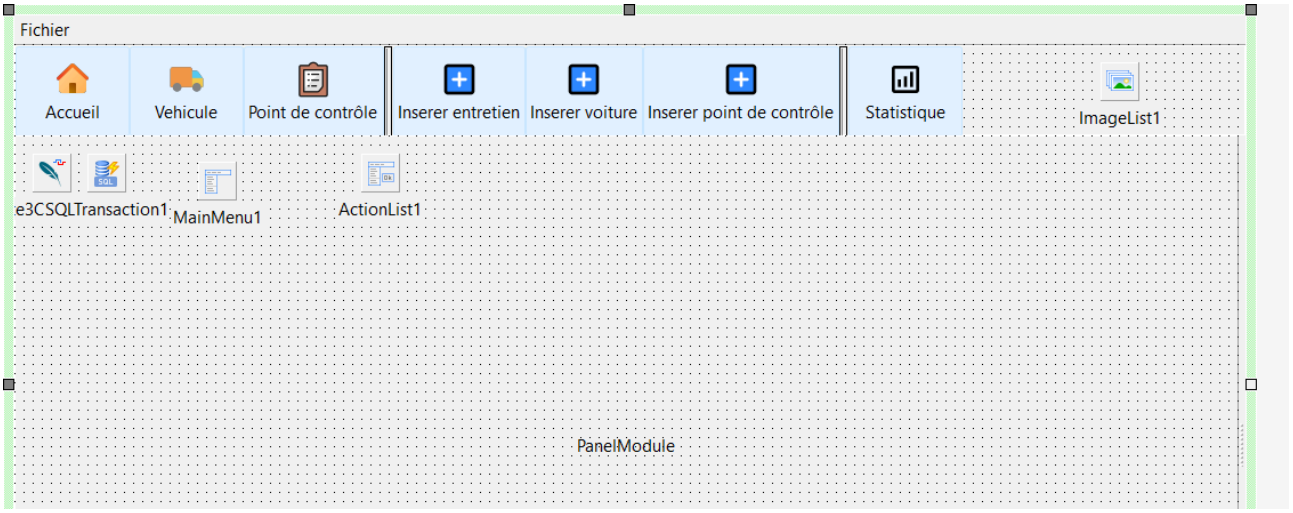



*Figure 0.1 : Conception de l'interface de navigation principale.*

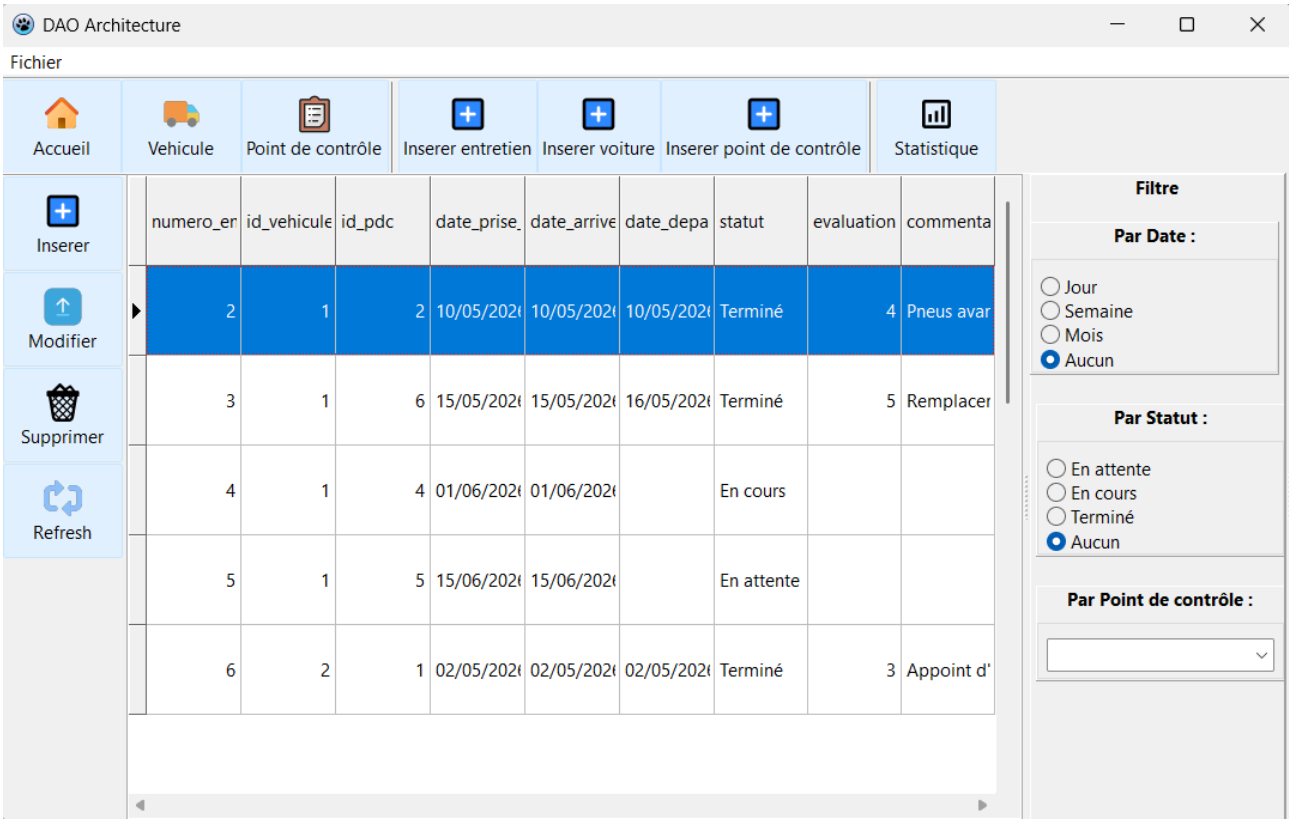



*Figure 0.2 : Vue de l'application en cours d'exécution.*

### Composants utilisés et motivation

| Composant | Type | Motivation du choix |
|---|---|---|
| **TToolBar** | Barre d'outils principale | Contient les gros boutons de navigation supérieure (Accueil, Véhicule, Point de contrôle, Statistiques, etc.) pour basculer d'un module à l'autre. |
| **TImageList** | Liste d'images | Utilisé pour stocker et attribuer les icônes de chaque bouton de la barre d'outils de navigation. |
| **TSQLConnector** | Connexion SQL | Assure la connexion physique à la base de données locale SQLite de l'application. |
| **TSQLTransaction** | Gestionnaire de transactions | Gère les transactions de la base de données (Validation/Annulation des modifications) pour assurer l'intégrité des données. |
| **TPanel (PanelModule)** | Panneau conteneur | Zone centrale vide configurée pour accueillir et afficher dynamiquement l'interface de chaque module sélectionné. |
| **TMainMenu / TActionList** | Menu et Actions | Gère le menu supérieur classique (Fichier) et centralise les actions de l'application. |

---

## 1. Fenêtre Principale (Accueil / Liste des entretiens)

### Fonctionnalités

La fenêtre d'accueil constitue le tableau de bord principal de l'application. Elle permet de :

- **Visualiser la liste des tâches d'entretien** : Affichage sous forme de tableau des entretiens avec leurs détails (véhicule, point de contrôle, dates, statut, évaluation, commentaire).
- **Filtrer les entretiens** :
  - *Par date* : filtrage rapide pour afficher les tâches du jour, de la semaine, du mois ou toutes les tâches (Aucun).
  - *Par statut* : filtrage par état d'avancement (En attente, En cours, Terminé, Aucun).

    - *Par point de contrôle* : sélection d'un point de contrôle spécifique via une liste déroulante.

- **Gérer les entretiens** : Insérer un nouvel entretien, modifier un entretien existant (ouvre la fiche détaillée), ou en supprimer un.
- **Rafraîchir la liste** : Met à jour les données affichées directement depuis la base SQLite.
- **Naviguer dans l'application** : Un menu supérieur (ToolBar) permet d'accéder aux autres modules (Véhicules, Points de contrôle, Statistiques).

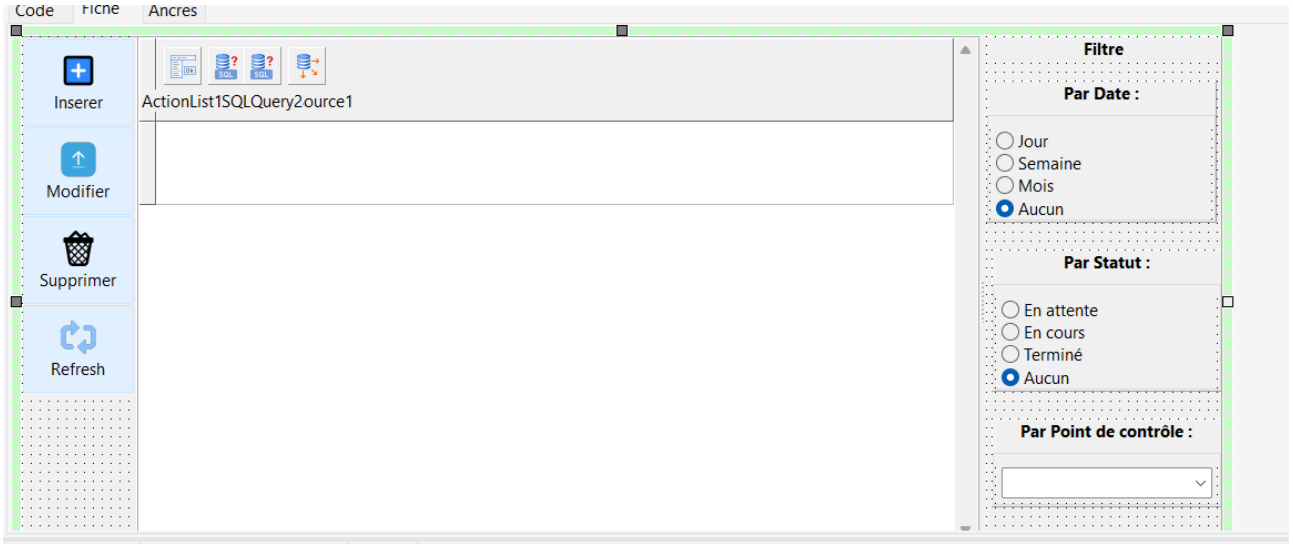



*Figure 1.1 : Conception du module d'accueil et de gestion des entretiens.*

### Composants utilisés et motivation

| Composant | Type | Motivation du choix |
|---|---|---|
| **TToolBar** | Barre d'outils | Permet une navigation rapide et intuitive entre les différents modules de l'application avec des icônes explicites. |
| **TDBGrid** | Grille de données | Composant idéal pour afficher de manière tabulaire et structurée les enregistrements de la base de données (table entretien avec jointures). |
| **TPanel** | Panneau conteneur | Utilisé pour structurer l'interface, notamment pour regrouper les filtres sur la droite (PanelFiltre, PanelDate, PanelStatut). |
| **TRadioGroup / TRadioButton** | Boutons radio | Choix pertinent pour les filtres mutuellement exclusifs (Date et Statut), permettant à l'utilisateur de comprendre d'un coup d'œil les options disponibles. |
| **TComboBox** | Liste déroulante | Utilisé pour le filtre par point de contrôle, car la liste peut être longue et dynamique. |
| **TSQLQuery / TDataSource** | Accès aux données | Composants invisibles essentiels pour lier la grille aux données SQLite et exécuter les requêtes de filtrage dynamiques. |

---

## 2. Fenêtre Gestion des Véhicules

### Fonctionnalités

Ce module est dédié à la gestion du parc de véhicules :

- **Lister les véhicules** : Affichage de tous les engins avec leurs caractéristiques (immatriculation, kilométrage, marque, modèle, couleur, poids).
- **Aperçu visuel** : Affichage de la photo du véhicule directement dans la grille de données.
- **Opérations CRUD** : Ajouter un nouveau véhicule, modifier les informations d'un véhicule existant, ou supprimer un véhicule (avec vérification des contraintes d'intégrité).

- **Rafraîchir la liste** : Met à jour le catalogue des véhicules.

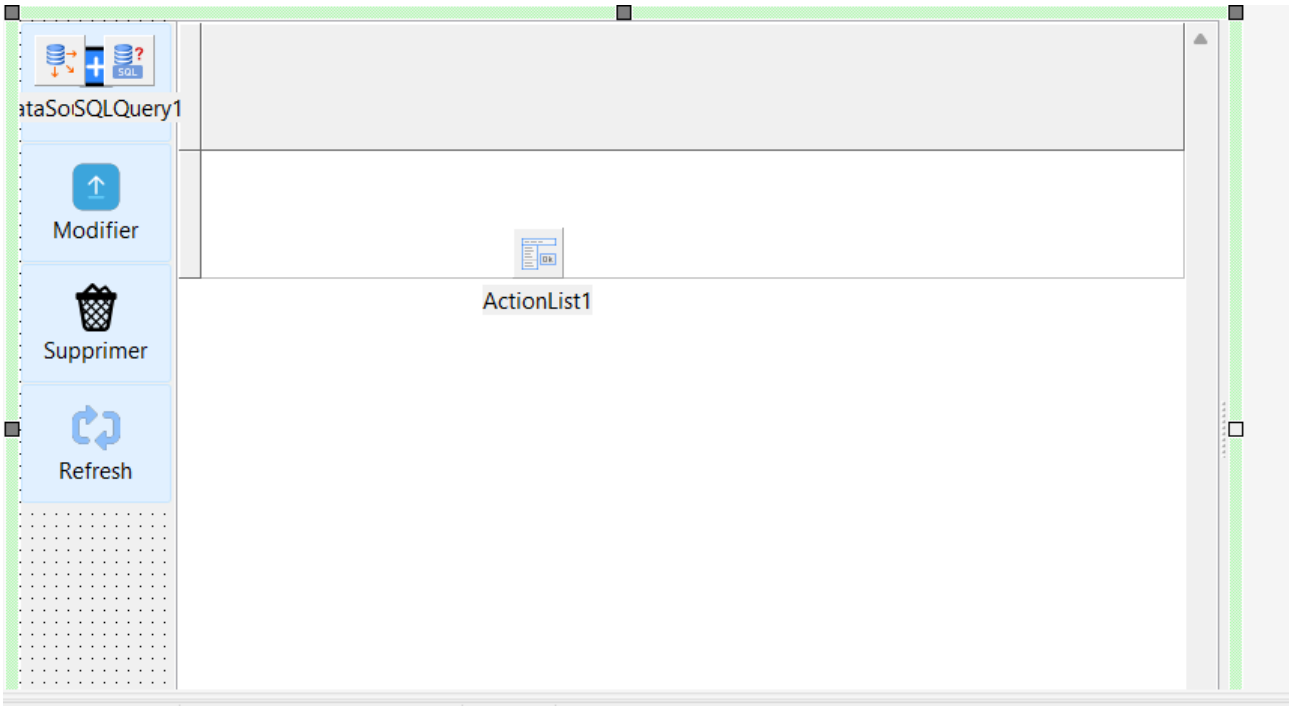



*Figure 2.1 : Conception du module de gestion des véhicules.*

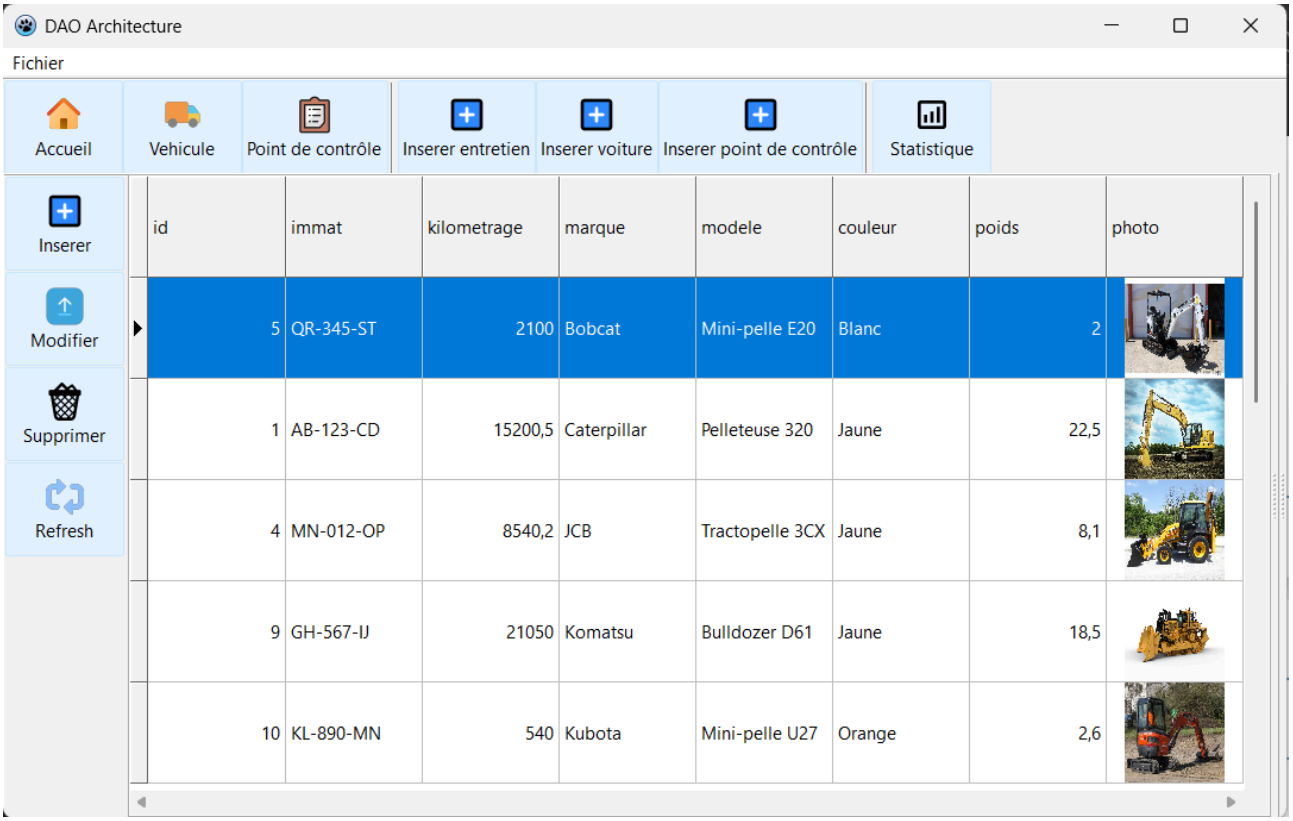



*Figure 2.2 : Liste des véhicules dans l'application.*

### Composants utilisés et motivation

| Composant | Type | Motivation du choix |
|---|---|---|
| **TDBGrid** | Grille de données | Affiche la liste des véhicules. L'événement OnDrawColumnCell est exploité pour dessiner dynamiquement les miniatures des photos. |
| **TToolBar** | Barre d'outils latérale | Regroupe les actions spécifiques à la gestion des véhicules (Insérer, Modifier, Supprimer, Refresh) de manière accessible. |
| **TSQLQuery / TDataSource** | Accès aux données | Permet de requêter la table des véhicules de la base SQLite et de lier les résultats directement à la grille de données. |

---

## 3. Fenêtre Gestion des Points de Contrôle

### Fonctionnalités

Ce module permet de paramétrer les différents types d'interventions possibles :

- **Lister les points de contrôle** : Affichage des points de contrôle avec leur nom, durée moyenne estimée et description.
- **Gérer le référentiel** : Créer de nouveaux points de contrôle, modifier les points existants, ou supprimer un point de contrôle (avec protection par clé étrangère).

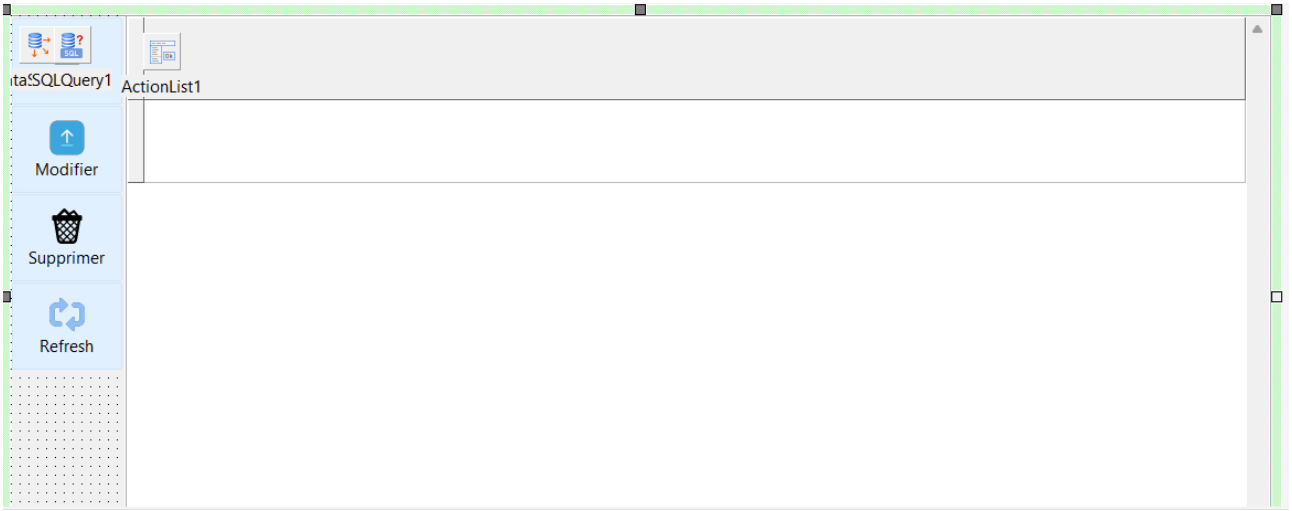



*Figure 3.1 : Conception du module des points de contrôle.*

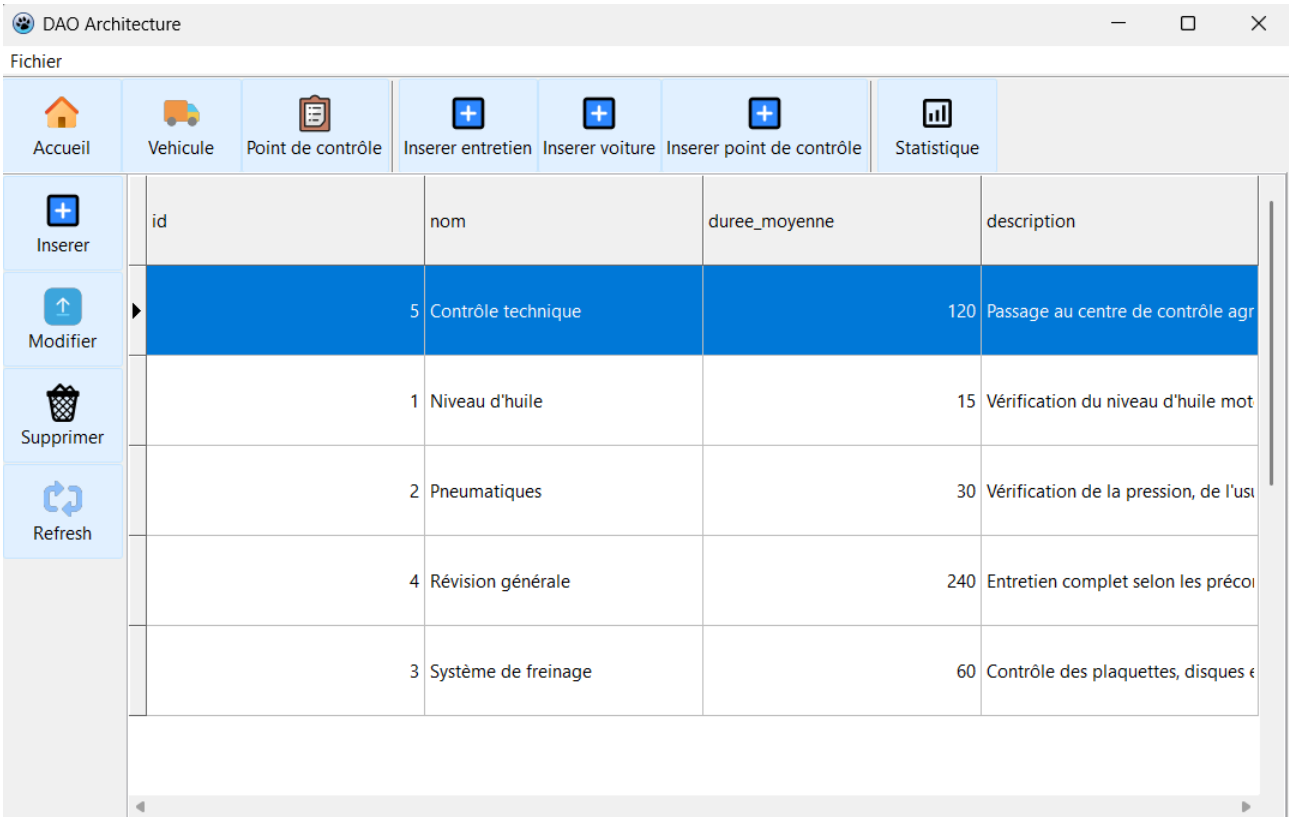



*Figure 3.2 : Référentiel des points de contrôle.*

### Composants utilisés et motivation

| Composant | Type | Motivation du choix |
|---|---|---|
| **TDBGrid** | Grille de données | Présentation claire et simple de la table point_de_controle. |
| **TToolBar** | Barre d'outils latérale | Fournit les boutons d'action d'édition standardisés (Insérer, Modifier, Supprimer, Refresh) pour maintenir une cohérence ergonomique. |
| **TSQLQuery / TDataSource** | Accès aux données | Exécute les requêtes de sélection pour lister et rafraîchir l'ensemble des points de contrôle disponibles. |

---

## 4. Formulaire d'Insertion d'un Entretien (DlgFormulaire1)

### Fonctionnalités

Ce formulaire modal permet de planifier une nouvelle tâche d'entretien :

- **Sélection du véhicule** : Choix rapide via une grille intégrée listant l'ensemble des véhicules disponibles.
- **Sélection du point de contrôle** : Choix rapide via une seconde grille listant les points de contrôle de référence.
- **Saisie des dates** : Renseignement des dates d'arrivée, de prise en charge et de départ prévu.
- **Automatisation** : Remplissage automatique et intelligent des dates selon le statut choisi.
- **Évaluation et Commentaire** : Saisie préventive d'une note d'évaluation et d'un texte descriptif associé.

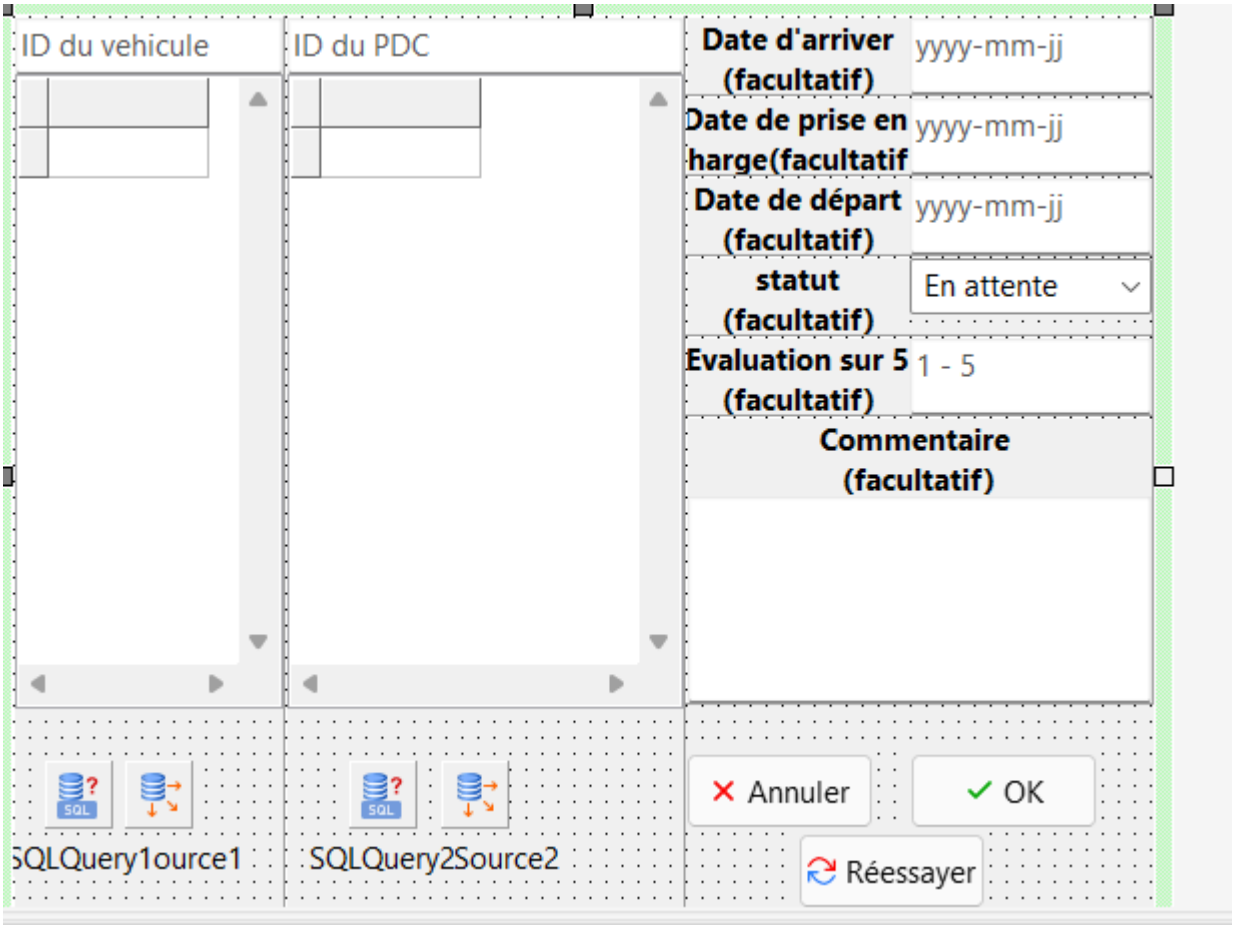



*Figure 4.1 : Conception du formulaire d'ajout d'entretien.*

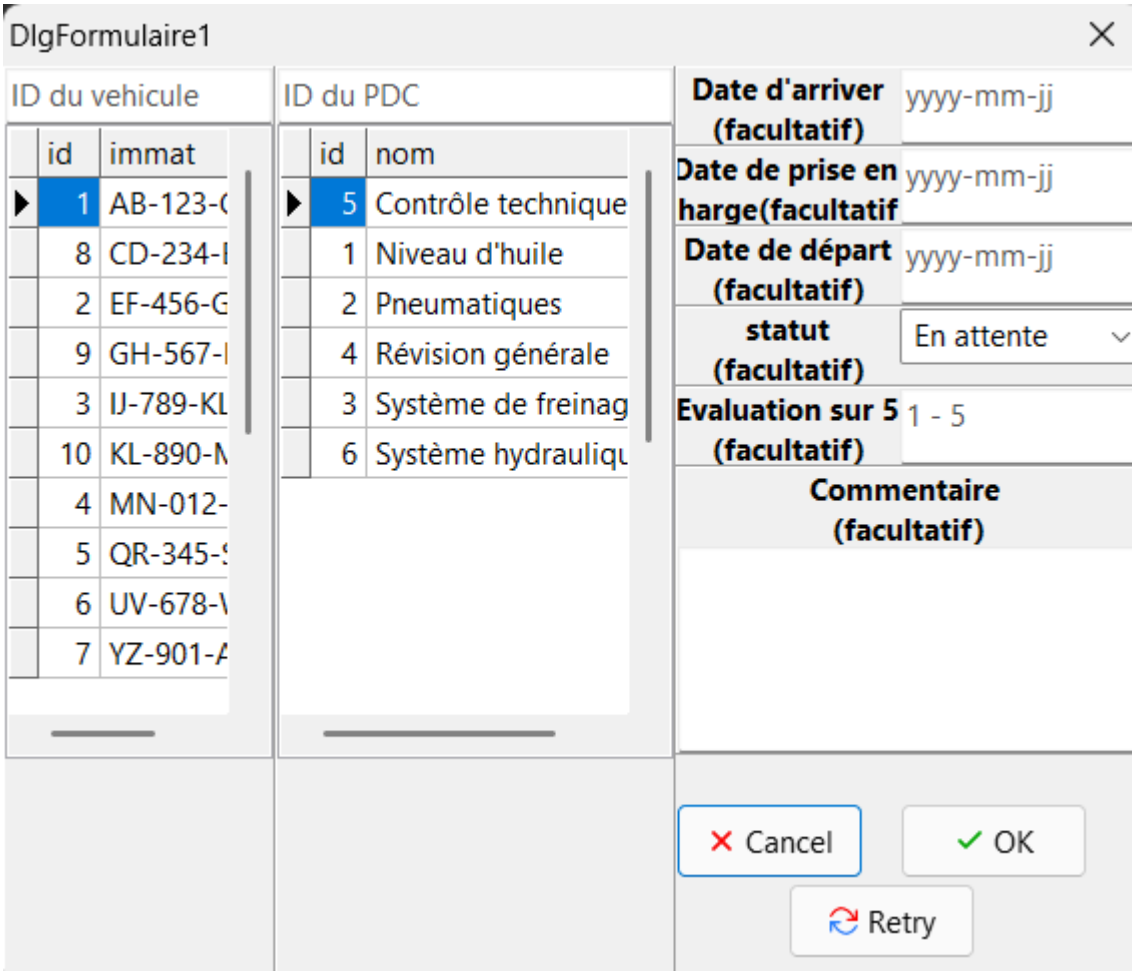



*Figure 4.2 : Saisie d’un nouvel entretien.*

### Composants utilisés et motivation

| Composant | Type | Motivation du choix |
|---|---|---|
| **TDBGrid (x2)** | Grilles de données | Utilisées pour la sélection visuelle du véhicule et du point de contrôle, plus intuitif qu'une simple liste déroulante. |
| **TEdit** | Champs de saisie | Utilisés pour la saisie guidée des dates et de l'évaluation. |
| **TComboBox** | Liste déroulante | Restreint le choix du statut aux valeurs autorisées et normées (En attente, En cours, Terminé). |
| **TMemo** | Zone de texte multiligne | Permet la saisie d'un commentaire détaillé sans restriction arbitraire de longueur. |
| **TBitBtn** | Boutons avec icônes | Actions classiques de validation (OK), annulation (Cancel) et réinitialisation (Retry) avec un retour visuel clair. |
| **TSQLQuery** | Accès aux données | Indispensable pour exécuter la requête d'insertion (INSERT INTO) du nouvel entretien directement dans la base. |

---

## 5. Fiche Détaillée d'un Entretien (DlgFormulaire4)

### Fonctionnalités

Cette fenêtre affiche les détails complets d'une tâche planifiée et permet son édition ou sa clôture :

- **Consultation globale** : Affichage de toutes les données associées à l'entretien et rappel de la photo de l'engin concerné.
- **Suivi par statut** : Répartition visuelle et claire des informations à travers des blocs de suivi (À effectuer, En cours, Terminé).
- **Validation des dates** : Contrôle logique intégré empêchant une date de départ antérieure à la date d'arrivée.
- **Génération HTML** : Création automatique d'une fiche récapitulative au format HTML et ouverture dans le navigateur.

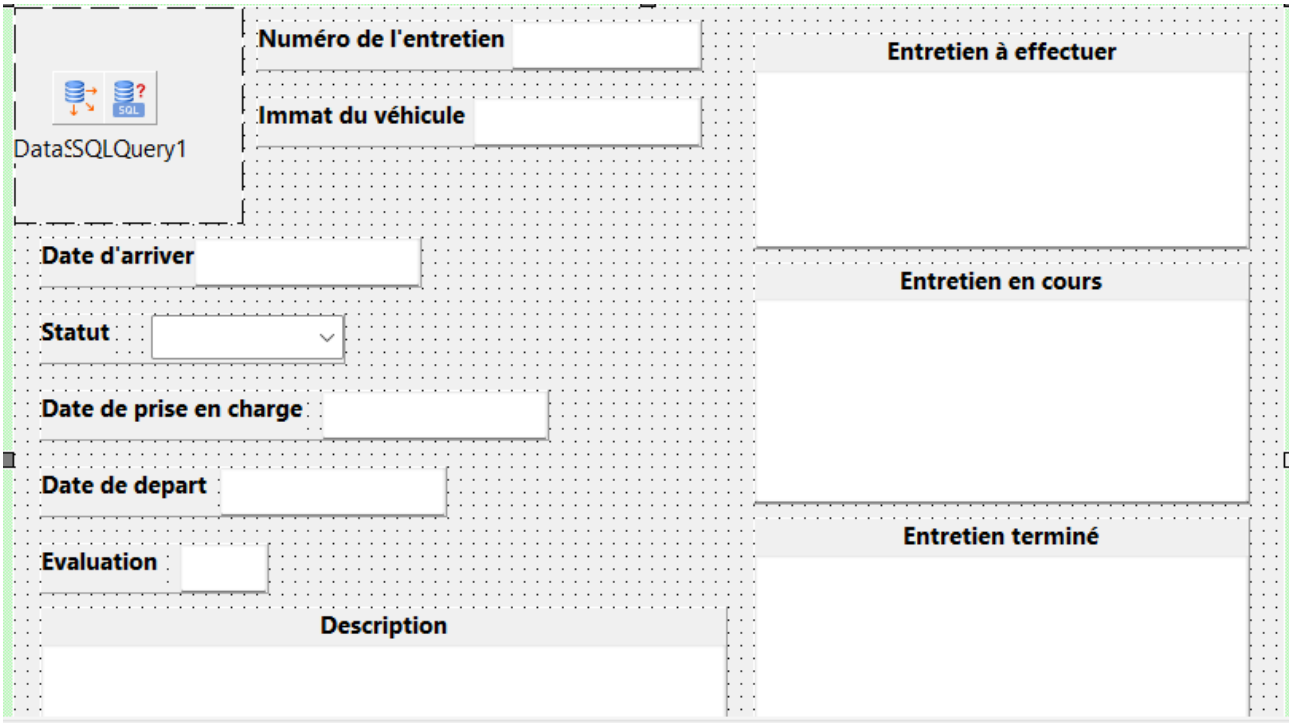



*Figure 5.1 : Conception de la fiche détaillée.*

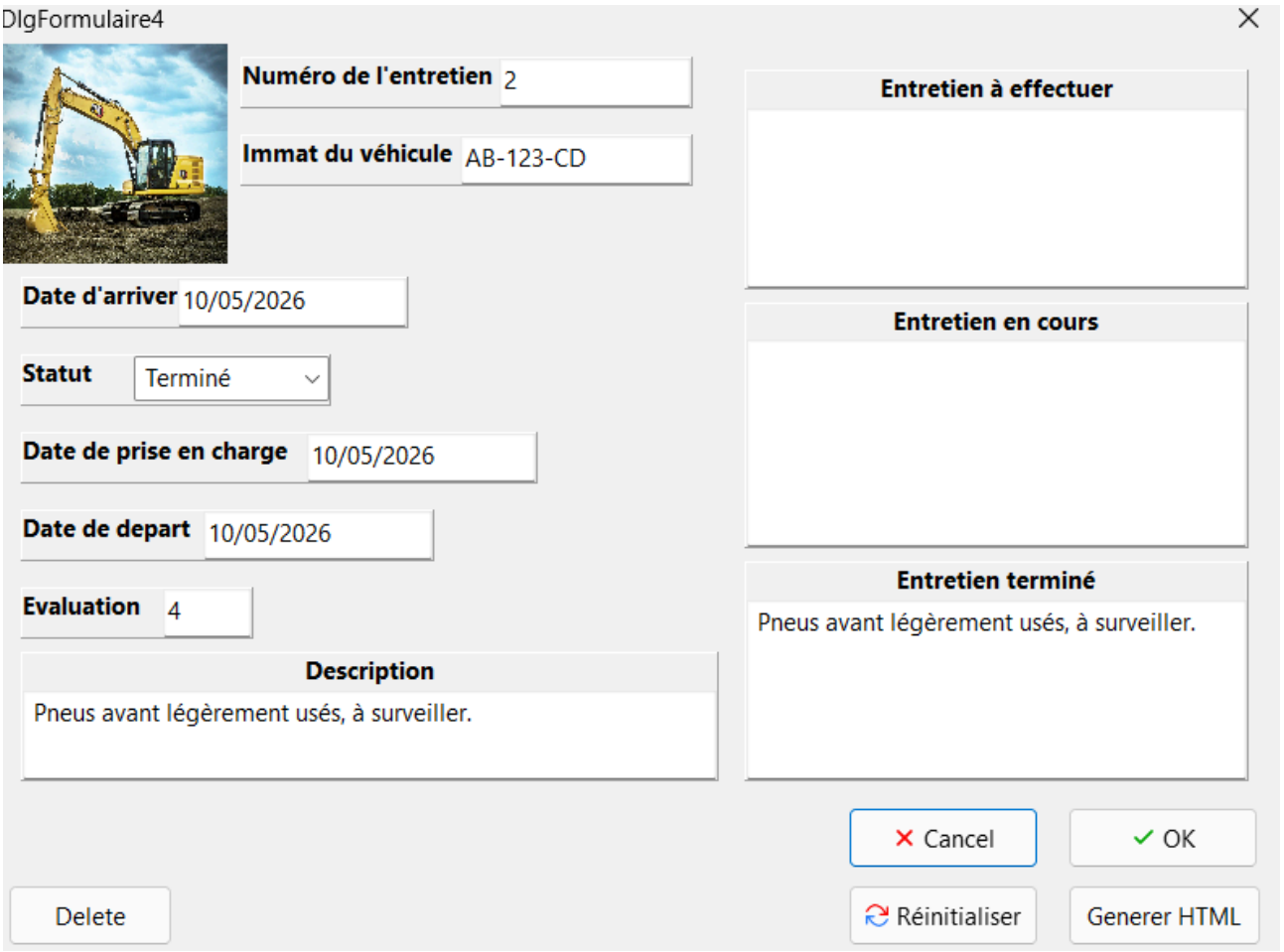



*Figure 5.2 : Édition et suivi d'un entretien en cours.*

### Composants utilisés et motivation

| Composant | Type | Motivation du choix |
|---|---|---|
| **TImage** | Afficheur d'image | Rappel visuel du véhicule concerné par l'intervention en affichant sa photo d'identité. |
| **TEdit / TComboBox** | Champs de saisie | Permettent la modification et la mise à jour rapide des dates de suivi, du statut et de l'évaluation. |
| **TMemo (x4)** | Zones de texte | Utilisés pour la description générale et pour les trois blocs de suivi visuel, offrant un espace de lecture confortable. |
| **TButton / TBitBtn** | Boutons d'action | Déclenchent les opérations spécifiques, notamment la suppression ou la génération HTML. |
| **TSQLQuery** | Accès aux données | Gère l'exécution des requêtes de mise à jour (UPDATE) ou de retrait (DELETE) de l'enregistrement. |

---

## 6. Formulaire d'Insertion d'un Véhicule (DlgFormulaire2)

### Fonctionnalités

Ce formulaire permet d'ajouter un nouvel engin au parc de l'entreprise :

- **Saisie des caractéristiques** : Renseignement précis de l'immatriculation, du kilométrage actuel, du poids, de la marque, du modèle et de la couleur.
- **Gestion de la photo** : Sélection d'une image sur le disque local. L'image est copiée et renommée automatiquement dans le sous-dossier photos de l'application.
- **Validation de saisie** : Vérification de la présence des champs obligatoires et de la conformité des formats numériques.

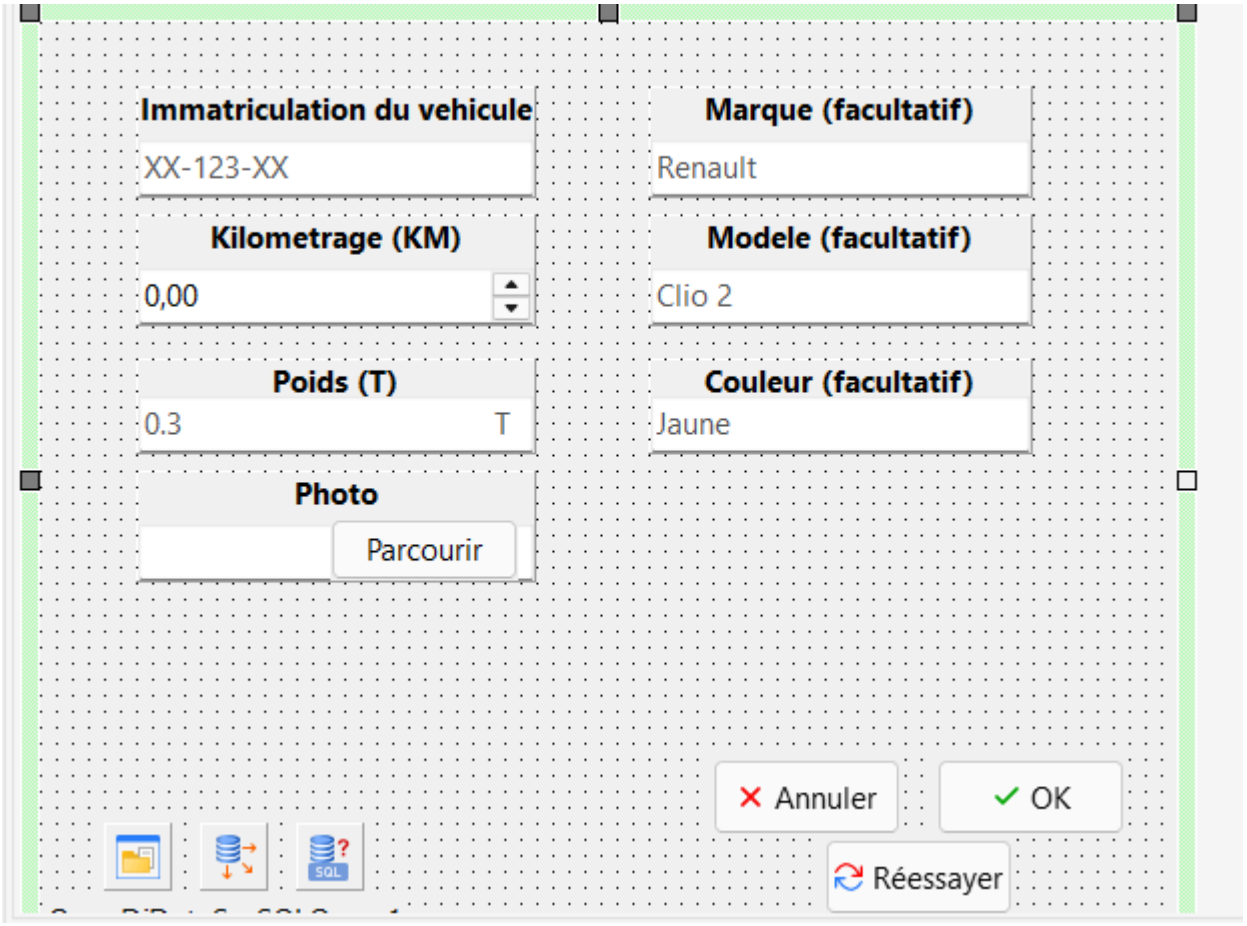



*Figure 6.1 : Conception du formulaire d'ajout de véhicule.*

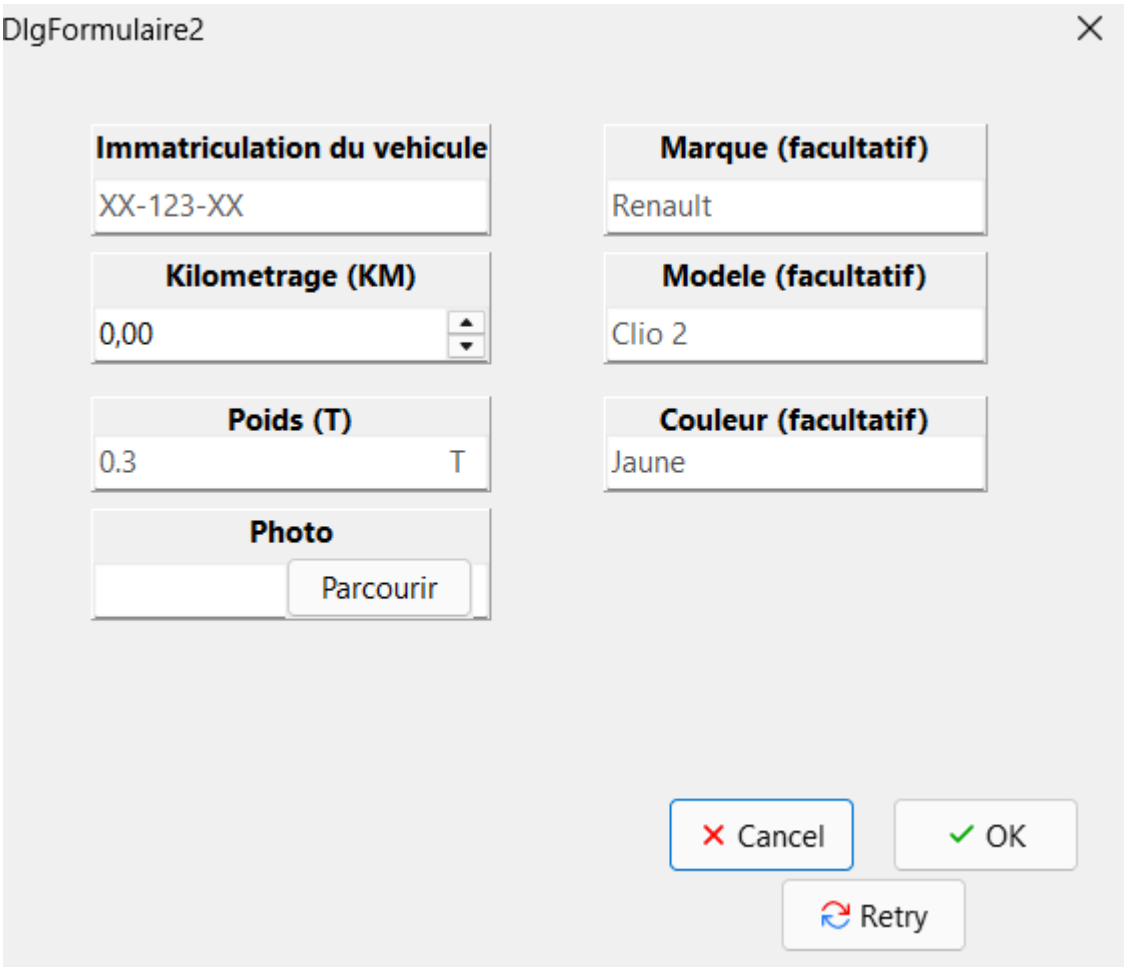



*Figure 6.2 : Ajout d'un nouveau véhicule au parc.*

### Composants utilisés et motivation

| Composant | Type | Motivation du choix |
|---|---|---|
| **TEdit** | Champs de texte | Saisie standard et efficace des informations textuelles (immatriculation, marque, modèle, couleur). |
| **TFloatSpinEdit** | Champ numérique | Idéal pour le kilométrage et le poids, empêche la saisie de caractères non numériques. |
| **TOpenDialog** | Boîte de dialogue | Permet à l'utilisateur de parcourir son système de fichiers pour sélectionner la photo. |
| **TBitBtn** | Boutons avec icônes | Actions d'exploration (Parcourir) et boutons de validation globale du formulaire. |
| **TSQLQuery** | Accès aux données | Utilisé pour injecter le nouveau véhicule dans la table correspondante via une requête SQL d'insertion. |

---

## 7. Formulaire de Mise à Jour d’un Véhicule (DlgFormulaire5)

### Fonctionnalités

Ce formulaire permet de modifier les caractéristiques d’un véhicule existant au sein du parc :

- **Modification des données** : Permet d’éditer l’immatriculation, le kilométrage, le poids, la marque, le modèle et la couleur.
- **Mise à jour de la photo** : Permet de remplacer l’image existante par une nouvelle photo via un sélecteur de fichiers.

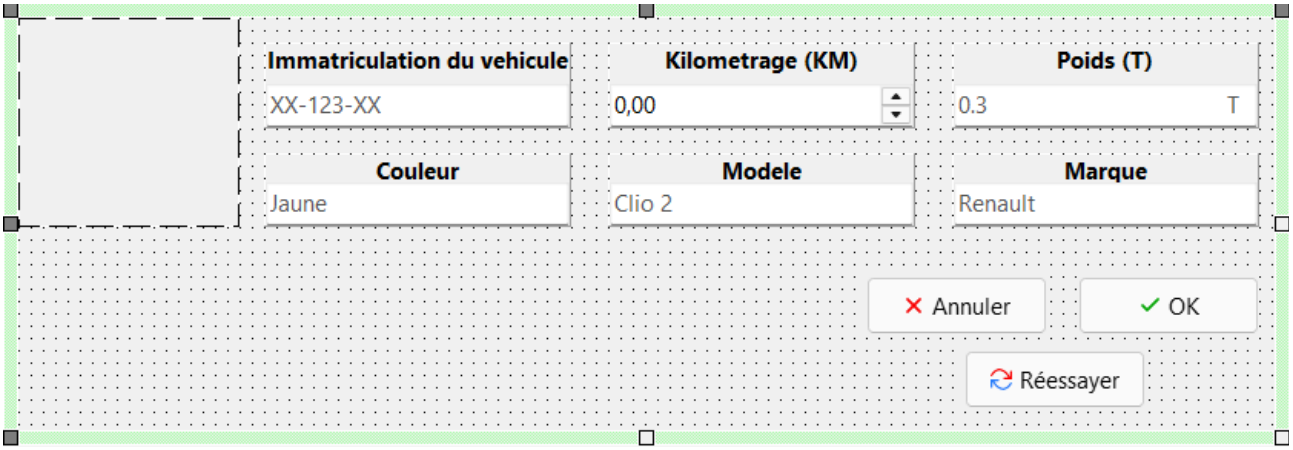



*Figure 7.1 : Conception du formulaire de mise à jour de véhicule.*

### Composants utilisés et motivation

| Composant | Type | Motivation du choix |
|---|---|---|
| **TEdit** | Champs de texte | Saisie et pré-remplissage des informations textuelles du véhicule pour modification. |
| **TFloatSpinEdit** | Champ numérique | Permet l’ajustement précis et sécurisé du kilométrage mis à jour et du poids de l’engin. |
| **TOpenDialog** | Boîte de dialogue | Permet de parcourir le disque pour sélectionner un nouveau fichier d’image. |
| **TBitBtn** | Boutons avec icônes | Boutons standards de validation des modifications (OK) ou d’annulation (Cancel). |

---

## 8. Formulaire d’Insertion d’un Point de Contrôle (DlgFormulaire3)

### Fonctionnalités

Formulaire épuré pour définir un nouveau type d’intervention dans le référentiel de l’atelier :

- **Saisie des informations** : Saisie du nom qualitatif du point de contrôle, de sa durée moyenne estimée (en minutes) et d’un commentaire.
- **Validation stricte** : Contrôle de la présence du nom (champ obligatoire) et de la cohérence de la durée (valeur entière positive).

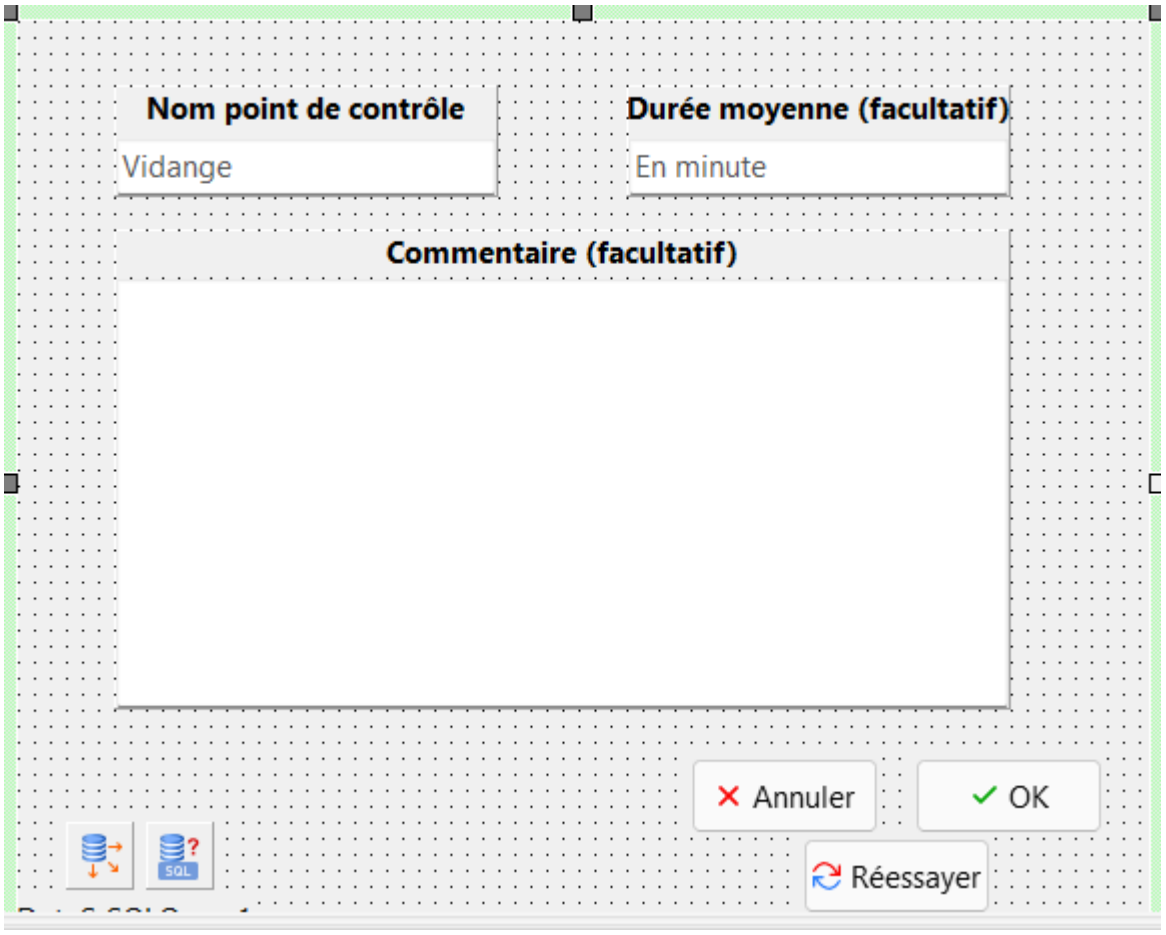



*Figure 8.1 : Conception du formulaire d’ajout de point de contrôle.*

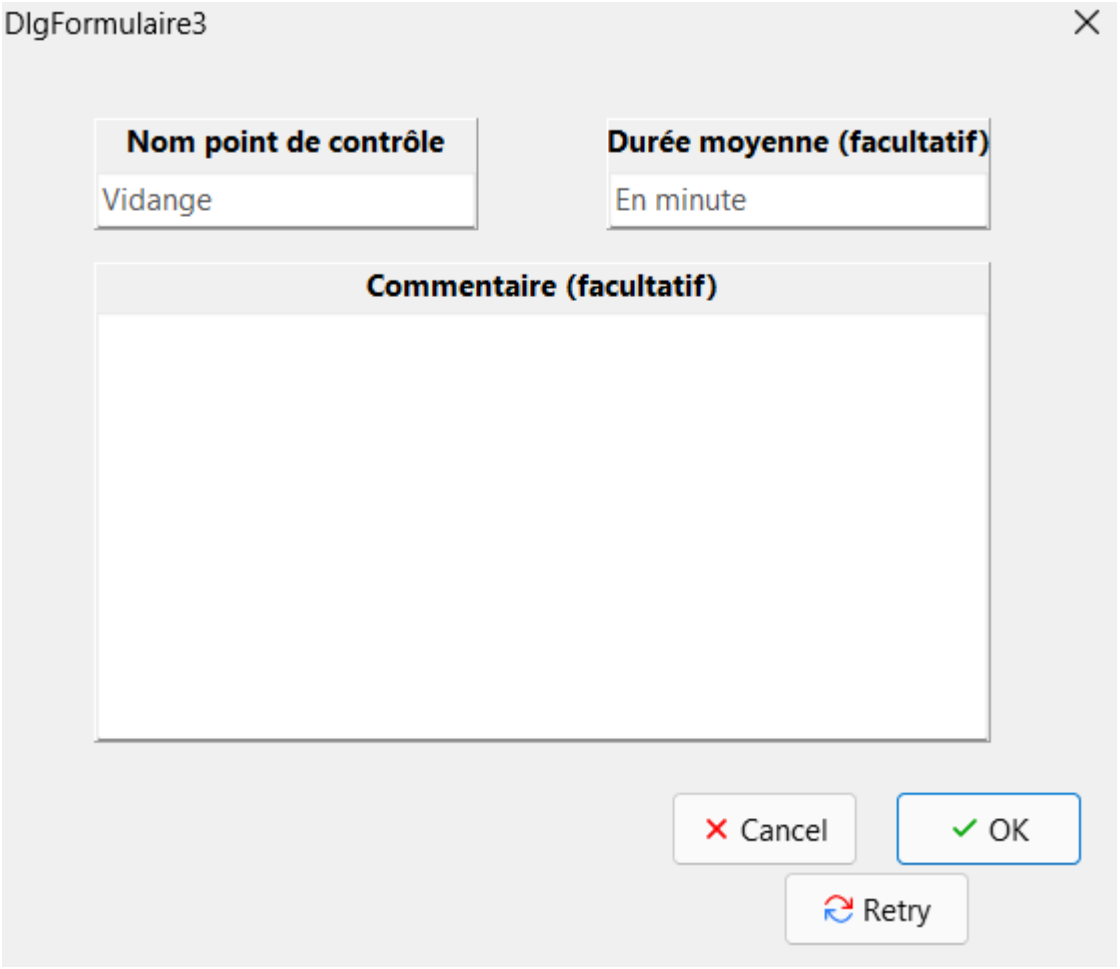



*Figure 8.2 : Création d'un nouveau point de contrôle.*

### Composants utilisés et motivation

| Composant | Type | Motivation du choix |
|---|---|---|
| **TEdit** | Champs de texte | Saisie rapide du nom du point de contrôle et de sa durée moyenne estimée. |
| **TMemo** | Zone de texte multiligne | Espace suffisant pour détailler les procédures ou commentaires spécifiques liés à ce point de contrôle. |
| **TBitBtn** | Boutons avec icônes | Actions de validation, annulation et réinitialisation rapide du formulaire. |
| **TSQLQuery** | Accès aux données | Utilisé pour envoyer et enregistrer la requête d'insertion du nouveau point de contrôle. |

---

## 9. Formulaire de Mise à Jour d’un Point de Contrôle (DlgFormulaire6)

### Fonctionnalités

Formulaire permettant l’édition d’un point de contrôle existant dans la base de données :

- **Édition du référentiel** : Permet d’ajuster le libellé, la durée moyenne de l’opération ainsi que les consignes ou notes associées.

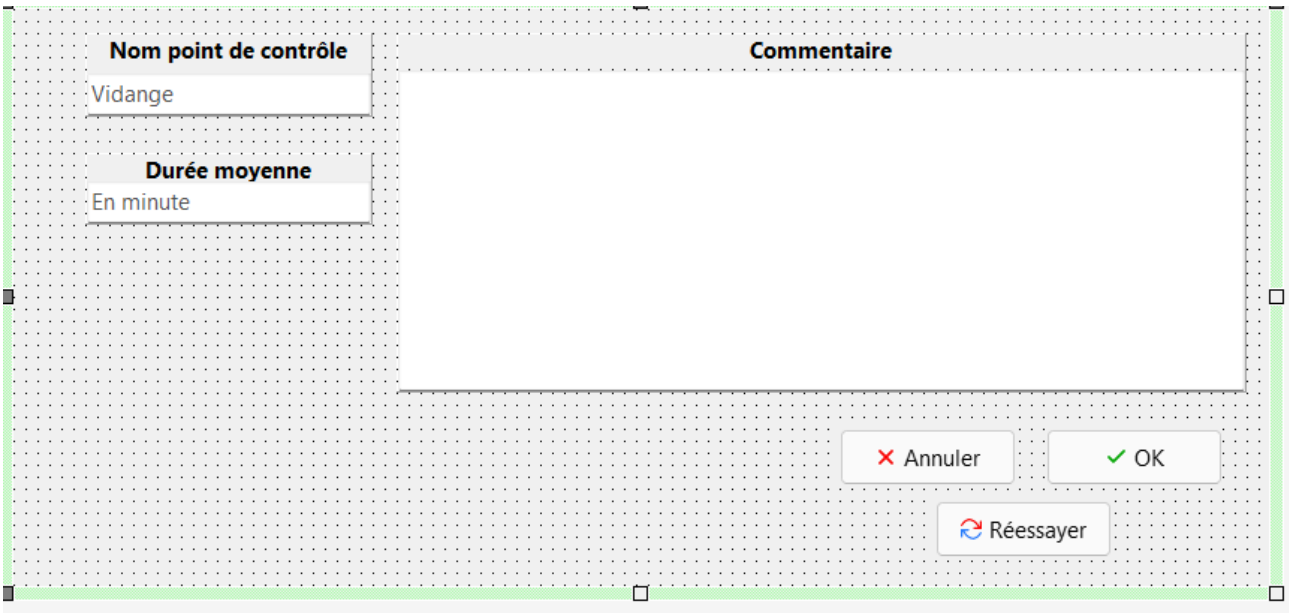



*Figure 9.1 : Conception du formulaire de mise à jour de point de contrôle.*

### Composants utilisés et motivation

| Composant | Type | Motivation du choix |
|---|---|---|
| **TEdit** | Champs de texte | Modification du nom et de la durée du point de contrôle sélectionné. |
| **TMemo** | Zone de texte multiligne | Mise à jour des descriptions textuelles et des consignes d’atelier. |
| **TBitBtn** | Boutons avec icônes | Validation des modifications ou fermeture sans enregistrer (OK, Cancel). |

---

## 10. Fenêtre des Statistiques

### Fonctionnalités

Ce module répond au besoin d'analyse de l'activité globale de l'atelier mécanique :

- **Agrégation des données** : Calcul automatisé et en temps réel des indicateurs statistiques regroupés par véhicule et par mois calendaire.
- **Indicateurs clés de performance** : Affichage du nombre total de tâches prévues, du nombre exact de tâches réalisées, ainsi que des évaluations qualitatives minimale, maximale et moyenne calculées.
- **Tri et agencement** : Les lignes statistiques sont présentées et triées par mois décroissant, puis par marque de véhicule.

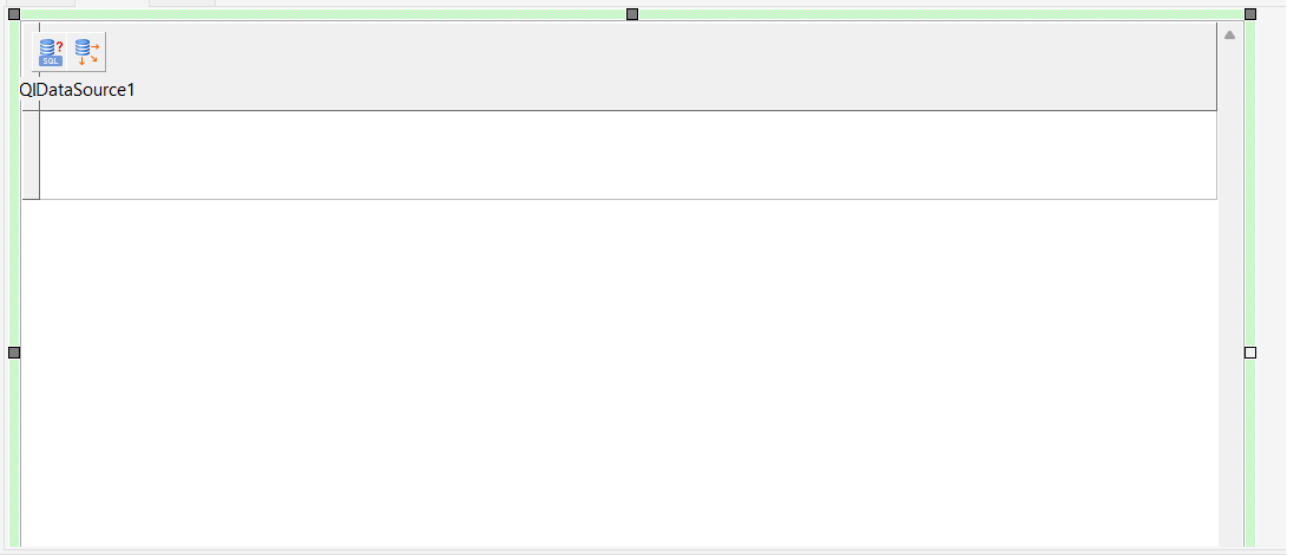



*Figure 10.1 : Conception du module statistique.*

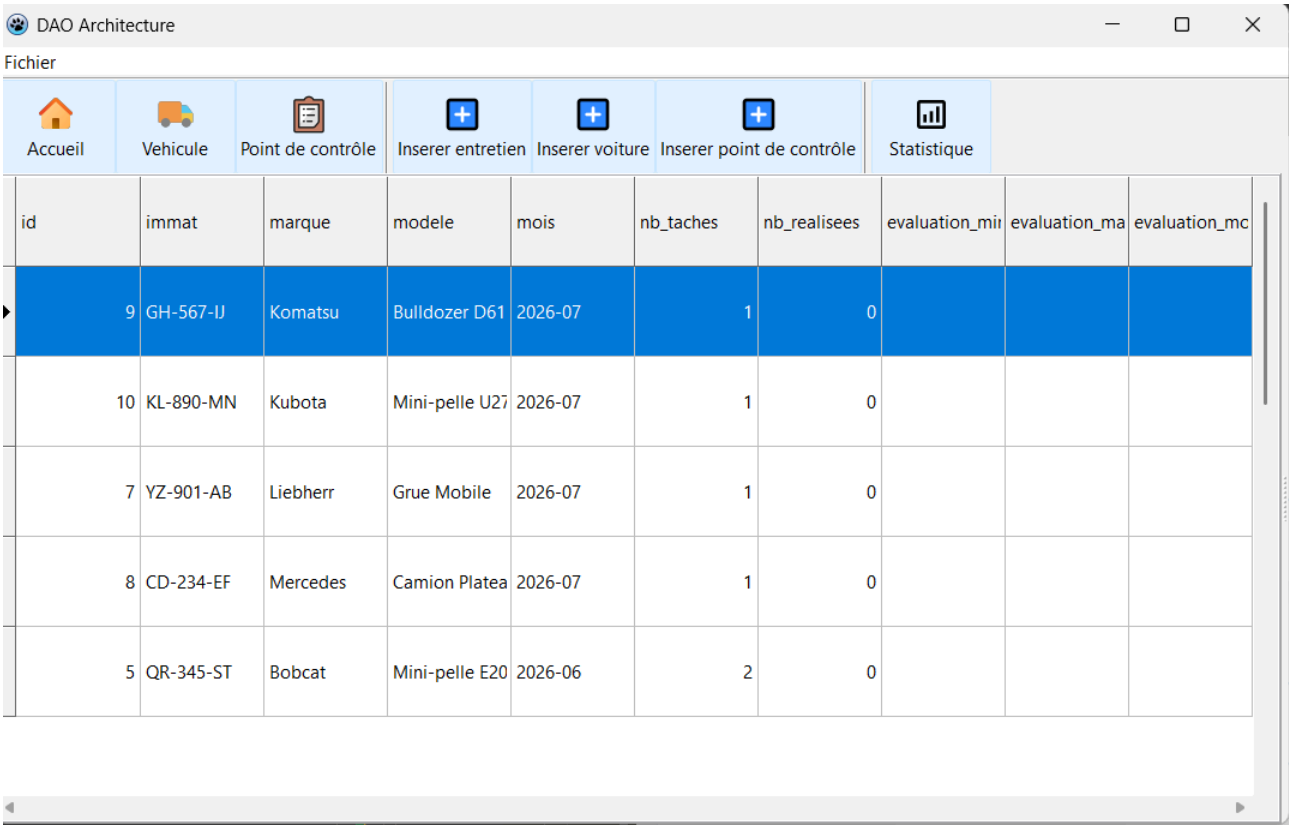



*Figure 10.2 : Vue des statistiques de l'application.*

### Composants utilisés et motivation

| Composant | Type | Motivation du choix |
|---|---|---|
| **TDBGrid** | Grille de données | Affichage optimal pour un tableau de bord statistique. La grille occupe tout l'espace disponible (Align = alClient). |
| **TSQLQuery** | Requête SQL | Composant central, il exécute une requête complexe utilisant des fonctions d'agrégation (COUNT, SUM, MIN, MAX, AVG) directement via le moteur SQLite. |

---

## Conclusion

L'application **MAINTANO** respecte l'ensemble des exigences fixées par le cahier des charges. L'interface utilisateur, développée avec la bibliothèque LCL de Lazarus, est structurée de manière hautement modulaire (utilisation de TFrame et séparation par Unit). Les choix de composants favorisent continuellement l'ergonomie globale tout en assurant la parfaite robustesse et portabilité de l'application grâce à l'adossement à une base de données SQLite locale.